Solución Ejercicio

1. Introducción y Alcance General

Este documento presenta la propuesta arquitectónica para el nuevo sistema de banca por internet de la entidad BP. El sistema permitirá a los clientes consultar el histórico de sus movimientos, realizar pagos y ejecutar transferencias entre cuentas propias e interbancarias con alta disponibilidad y baja latencia.

Dado que la información del cliente reside actualmente en dos orígenes distintos (una Plataforma Core con datos básicos, productos y movimientos, y un Sistema Independiente que entrega el detalle complementario del cliente), se plantea una arquitectura basada en microservicios desacoplados en la nube (AWS), utilizando una capa de integración con API Gateway, caché distribuida para clientes frecuentes y procesamiento asíncrono por colas para cumplir con el envío obligatorio de notificaciones y el registro en la base de datos de auditoría sin degradar el tiempo de respuesta.

2. Decisiones Arquitectónicas y Justificación Teórica

Siguiendo las buenas prácticas de diseño, para cada componente crítico se evaluaron al menos dos alternativas técnicas antes de tomar una decisión.

Componente / Capa	**Opciones Evaluadas**	**Elección**	**Justificación Técnica y Trade-offs**
1. Aplicación Web (SPA)	1. Angular (v17+) 2. React.js	Angular	En banca se requiere una estructura estricta y estandarizada para facilitar el mantenimiento entre distintos equipos. Angular ofrece inyección de dependencias nativa, tipado fuerte con TypeScript, Guards de rutas e

Componente / Capa	Opciones Evaluadas	Elección	Justificación Técnica y Trade-offs
			HttpInterceptors ideales para adjuntar tokens OAuth 2.0 y manejar refresco de sesiones de forma centralizada. React depende de librerías de terceros para enrutamiento y estado, aumentando el riesgo de inconsistencias.
2. Aplicación Móvil Multiplataforma	1. Flutter (Dart) 2. React Native	Flutter	Se elige Flutter por dos motivos clave: rendimiento en cámara/biometría y seguridad. Al compilar directo a código nativo ARM (AOT) sin usar un puente JS como React Native, la captura de video para reconocimiento facial en el Onboarding no sufre caídas de frames. Además, el binario compilado en Dart es más difícil de vulnerar mediante ingeniería inversa que un bundle de

Componente / Capa	Opciones Evaluadas	Elección	Justificación Técnica y Trade-offs
			JavaScript.
3. Capa de Integración y API Gateway	1. API Gateway directo 2. API Gateway + Patrón BFF (Backend for Frontend)	API Gateway + BFF	El API Gateway (AWS API Gateway) maneja seguridad perimetral, validación JWT y Rate Limiting. Detrás se coloca un BFF para adaptar las respuestas según el canal: la app móvil requiere tramas más ligeras que la web, y el BFF permite orquestar en una sola llamada la unión de datos entre el Core y el Sistema Independiente de detalle.
4. Stack de Microservicios	1. .NET 8 (C#) 2. Node.js (Express/NestJS)	.NET 8 (C#)	Para transacciones financieras concurrentes, el motor de .NET 8 ofrece mejor rendimiento multihilo, menor huella de memoria en contenedores Linux y tipado seguro para cálculos monetarios (decimal), además de integración nativa con

Componente / Capa	Opciones Evaluadas	Elección	Justificación Técnica y Trade-offs
			políticas de resiliencia (Polly) y comunicación gRPC.
5. Protocolos de Comunicación	1. REST (JSON/HTTPS) 2. gRPC (HTTP/2) 3. AMQP / Colas (SNS/SQS o RabbitMQ)	Esquema Híbrido	Se usa HTTPS/REST desde la SPA y App Móvil hacia el API Gateway por compatibilidad estándar; gRPC en la red interna entre el BFF y los microservicios para reducir latencia gracias a la serialización binaria Protobuf; y mensajería asíncrona (AMQP/SQS) para notificaciones y auditoría, evitando que una lentitud en el servidor de correo bloquee una transferencia.
6. Base de Datos de Auditoría	1. BD Relacional (SQL Server / PostgreSQL) 2. BD NoSQL Documental / Key-Value	AWS DynamoDB (NoSQL)	La auditoría registra cada clic y acción del cliente, generando un volumen de escritura muy alto con esquemas variables

Componente / Capa	Opciones Evaluadas	Elección	Justificación Técnica y Trade-offs
	(DynamoDB / MongoDB)		según el evento. Una base relacional sufriría bloqueos de índices a largo plazo; DynamoDB permite escrituras masivas con latencia de milisegundos, particionado por cliente/fecha y exportación automática a almacenamiento frío.

3. Arquitectura de Autenticación (OAuth 2.0) y Onboarding Biométrico

3.1. Selección del Flujo OAuth 2.0

La entidad BP ya dispone de un gestor de identidad compatible con OAuth 2.0 (ej. Keycloak, AWS Cognito o Azure AD B2C). Dado que tanto la SPA en Angular como la App Móvil en Flutter son clientes públicos (no pueden guardar un *client_secret* de forma segura porque el código se ejecuta en el dispositivo del usuario), se evaluaron los flujos del estándar:

- Opciones evaluadas: *Authorization Code Flow con PKCE* vs. *Implicit Flow* vs. *Resource Owner Password Credentials (ROPC)*.
- Recomendación: Configurar OAuth 2.0 Authorization Code Flow con PKCE (Proof Key for Code Exchange - RFC 7636) sobre OpenID Connect (OIDC).
- Justificación: El flujo *Implicit* está obsoleto porque expone el token directamente en la URL del navegador. El flujo *ROPC* (enviar usuario y clave directo por API) rompe el principio de delegación y complica integrar biometría o MFA. Con PKCE, la aplicación genera un código verificador dinámico (*code_verifier*) y envía su hash SHA-256 (*code_challenge*) al iniciar sesión; cuando el servidor devuelve el código de autorización temporal, solo la app que generó el verificador original en memoria puede canjearlo por el JWT (*Access Token* de corta vida de 5 min y

Refresh Token rotativo). Esto neutraliza ataques de intercepción de código en móviles.

3.2. Flujo de Onboarding con Reconocimiento Facial y Login Posterior

Para el registro de nuevos clientes desde la app móvil y sus accesos posteriores se define el siguiente diseño:

1. Onboarding y Prueba de Vida (Liveness): El usuario nuevo abre la App Móvil e inicia el registro. La app utiliza el SDK de una herramienta probada en la industria como AWS Rekognition Face Liveness (o alternativas especializadas como Jumio / Incode). El SDK graba una secuencia corta aplicando cambios de luz para comprobar que es una persona viva y no una foto o video reproducido en otra pantalla (estándar ISO/IEC 30107-3).
2. Validación Documental y Creación de Cuenta: El *Microservicio de Onboarding* recibe la imagen facial y la foto de la cédula/documento, extrae los datos con OCR (AWS Textract) y valida la identidad contra el Registro Civil. Si el score biométrico supera el 98%, aprovisiona el usuario en el servidor OAuth 2.0 y registra la ficha base en el Core.
3. Ingresos Posteriores (Usuario/Clave, Huella o FaceID):
    - Usuario y Clave: Sigue el flujo OAuth 2.0 + PKCE acompañado de un segundo factor (OTP por SMS o token en app).
    - Huella Dactilar / Biometría Nativa: Por seguridad, la huella nunca viaja al backend. Se implementa el estándar FIDO2 / WebAuthn apoyado en el hardware seguro del teléfono (*Keystore* en Android y *Secure Enclave* en iOS). Al terminar el Onboarding y activar la huella, el teléfono guarda una llave privada criptográfica protegida por el lector biométrico y registra la llave pública en el servidor OAuth 2.0. En futuros logins, al poner el dedo en el lector, el chip desbloquea la llave privada y firma un reto criptográfico (*challenge*), obteniendo el token JWT en menos de 1 segundo.

4. Diseño de Servicios, Auditoría y Persistencia para Clientes Frecuentes

4.1. Catálogo de Microservicios

Partiendo de los 3 servicios base del negocio, se agregan 3 servicios adicionales para desacoplar cargas pesadas y mejorar los tiempos de respuesta:

- MS Consulta de Datos Básicos (Base): Obtiene la información general del cliente desde la Plataforma Core y, cuando la petición requiere nivel de detalle, consulta al Sistema Independiente y unifica la respuesta.
- MS Consulta de Movimientos (Base): Entrega el historial de transacciones filtrable y paginado.
- MS Transferencias y Pagos (Base): Procesa transferencias propias e interbancarias validando fondos, límites diarios y llaves de idempotencia para evitar transacciones duplicadas si el usuario presiona dos veces el botón.
- MS Onboarding Digital (Adicional): Aísla el procesamiento pesado de imágenes y llamadas a AWS Rekognition para

que un pico de registros nuevos no consuma el CPU de las transferencias.

- MS Orquestador de Notificaciones (Adicional): Consume los eventos de movimientos realizados y envía las alertas obligatorias por norma a través de 2 sistemas independientes:
    1. Canal 1 - Correo Electrónico: Integrado con AWS SES (con fallback a SendGrid).
    2. Canal 2 - Mensajería Móvil (SMS y Push): Integrado con Twilio para SMS transaccionales y Firebase Cloud Messaging (FCM) para notificaciones Push.
- MS Worker de Auditoría (Adicional): Consume eventos desde la cola y los persiste en la base de datos de auditoría sin frenar el flujo principal del cliente.

4.2. Propuesta Basada en Patrones para Auditoría y Clientes Frecuentes

Para relacionar el registro de auditoría con el mecanismo de persistencia rápida para clientes frecuentes sin acoplar el código, se combinan los siguientes patrones de diseño:

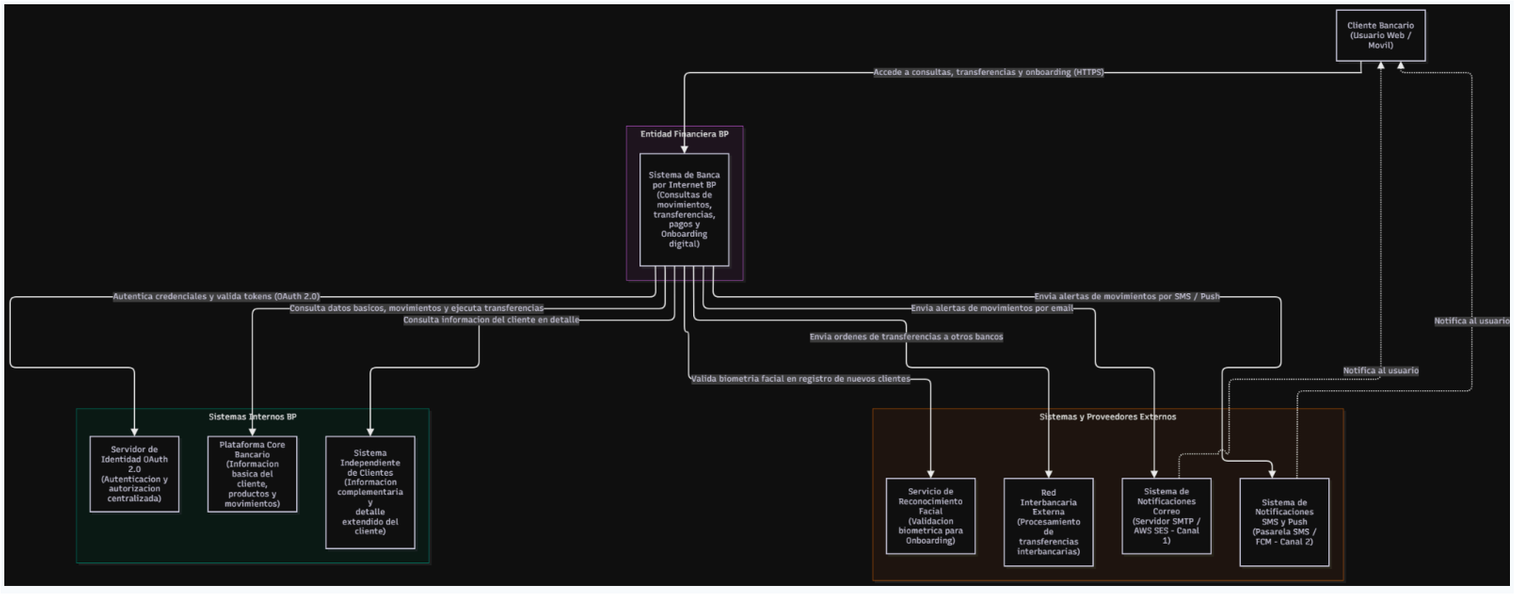
1. Patrón Decorator / Proxy + Cache-Aside (Persistencia de Clientes Frecuentes): En los servicios de consulta se implementa un componente FrequentClientCacheDecorator que envuelve al repositorio que llama al Core y al Sistema Independiente. Cada vez que un cliente autenticado realiza acciones, se incrementa un contador con expiración (TTL) en Redis (AWS ElastiCache). Si el usuario supera el umbral (ej. más de 3 accesos en la semana, clasificándose como *Cliente Frecuente*), su ficha consolidada (Core + Detalle) y sus últimos movimientos quedan persistidos en Redis. Las siguientes consultas se responden en menos de 10 ms sin tocar el Core bancario.
2. Patrón CQRS + Invalidación por Eventos: Se separan las consultas (que leen de la caché de clientes frecuentes) de los comandos transaccionales. Cuando el cliente realiza una transferencia o pago en el *MS de Transferencias*, este publica un evento de dominio que actualiza o invalida de inmediato la entrada en Redis para que el cliente frecuente nunca vea un saldo desactualizado.
3. Patrón Transactional Outbox + Observer (Pub/Sub) para Auditoría: Todas las peticiones que pasan por los microservicios generan un evento de auditoría (quién, qué acción, IP, dispositivo, timestamp y resultado). Para no perder ningún rastro si el broker de mensajes parpadea durante una transferencia, el servicio guarda la operación y el evento en una tabla *Outbox* en la misma transacción local, y un proceso los publica en el bus (RabbitMQ / AWS SNS-SQS). El *MS Worker de Auditoría* (Observer) toma el mensaje de la cola y lo guarda en DynamoDB en modo inmutable (solo inserción).

5. Diagramas de Arquitectura (Modelo C4 e Infraestructura)

5.1. Diagrama de Contexto (C4 - Nivel 1)

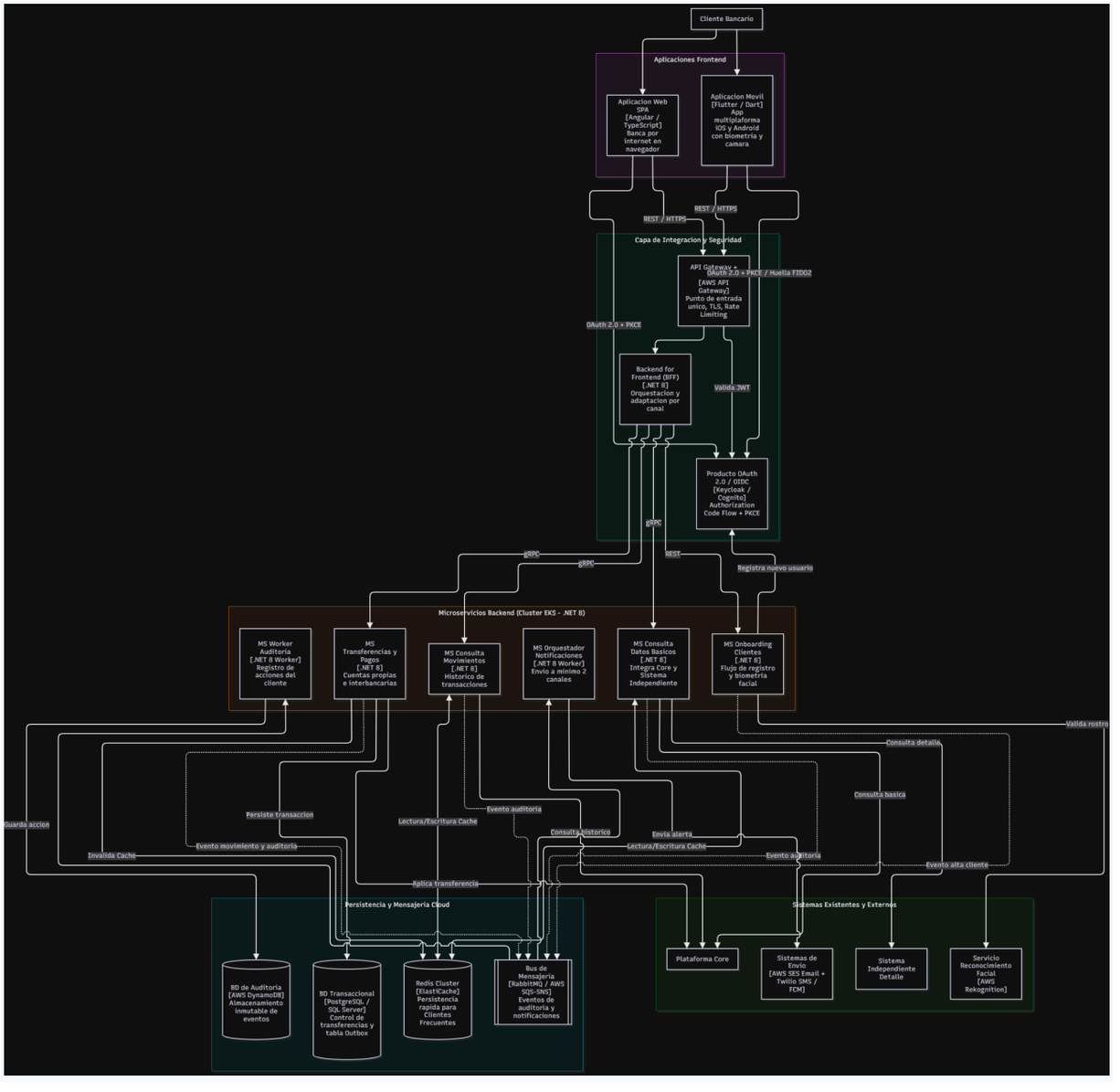
Muestra la vista general para usuarios no técnicos: el cliente bancario interactuando con el Sistema de Banca por Internet

BP, y las conexiones hacia los 2 sistemas de información internos (Core y Sistema Independiente de detalle), el servicio OAuth 2.0, el validador de reconocimiento facial y los 2 sistemas externos de notificación (Correo y SMS/Push).



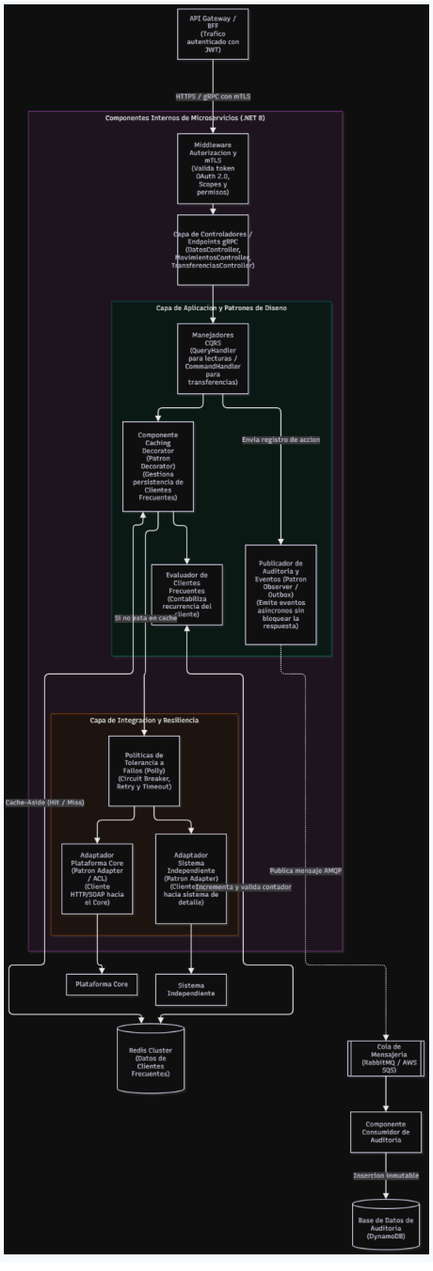
5.2. Diagrama de Contenedores (C4 - Nivel 2)

Muestra el detalle técnico de las aplicaciones (SPA Angular y App Móvil Flutter), el API Gateway con el BFF, los microservicios en .NET 8, la caché Redis para clientes frecuentes, el bus de mensajería y las bases de datos.



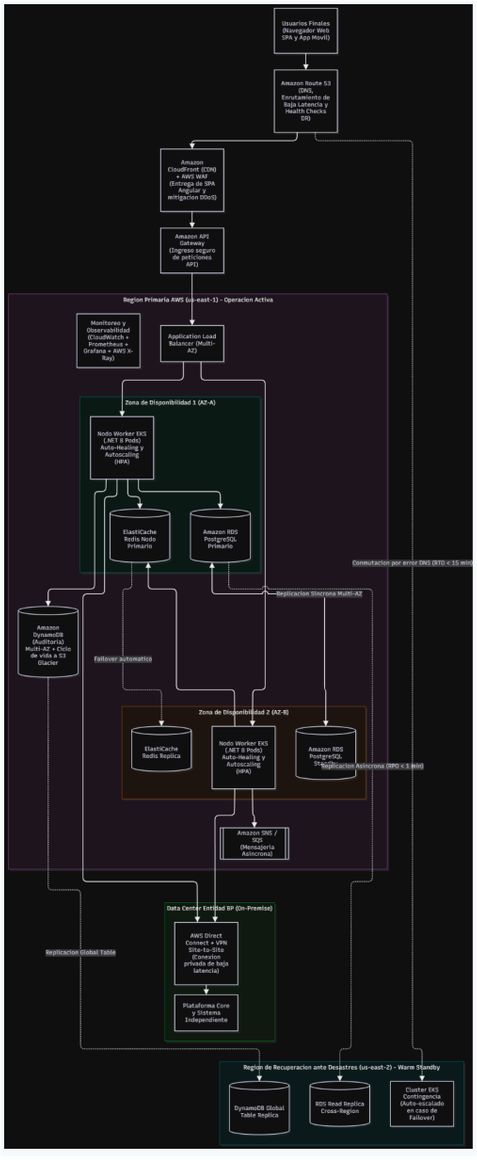
5.3. Diagrama de Componentes (C4 - Nivel 3)

Detalla la estructura interna de los servicios principales, destacando cómo interactúan los controladores, el Decorator de caché para clientes frecuentes, el publicador de eventos de auditoría, las políticas de resiliencia (Circuit Breaker) y los adaptadores hacia el Core y el Sistema Independiente usando protocolos seguros.



5.4. Diagrama de Infraestructura en Nube (AWS)

Representa el despliegue sobre AWS en múltiples zonas de disponibilidad (Multi-AZ), protección perimetral con WAF/CloudFront, clúster de Kubernetes (EKS) con auto-healing, enlace dedicado hacia el Data Center del banco y replicación hacia la región de contingencia (DR).



6. Consideraciones Normativas, Seguridad, HA, DR, Monitoreo y Costos

6.1. Elementos Normativos y Seguridad Bancaria

- Ley de Protección de Datos Personales (LOPDP / GDPR): Los datos sensibles y biométricos requieren consentimiento expreso durante el Onboarding. En los logs y en la base de auditoría se enmascaran números de cuenta y tarjetas (Data Masking). Todo dato se cifra en tránsito con TLS 1.3 y en reposo con AES-256 mediante llaves rotadas en AWS KMS.
- Normativa de Superintendencia de Bancos y Riesgo Operativo: Obligatoriedad de notificar al cliente por cada movimiento financiero (cubierto con el doble canal SES + Twilio/FCM) y mantener pistas de auditoría inalterables para peritaje forense.
- Seguridad en Capas (PCI-DSS / OWASP API Top 10): Uso de AWS WAF y Shield contra ataques DDoS e inyecciones; comunicación interna entre microservicios asegurada con mTLS (Mutual TLS sobre Service Mesh) y validación estricta de permisos (Scopes JWT) para impedir ataques BOLA (que un usuario consulte movimientos de otra cuenta cambiando el ID en la petición).

6.2. Alta Disponibilidad (HA), Tolerancia a Fallos, Auto-Healing y DR

- Alta Disponibilidad (HA): Despliegue en AWS (región primaria us-east-1 por menor latencia de red hacia la región andina) distribuyendo los nodos de EKS, instancias de Redis y bases de datos RDS en 3 Zonas de Disponibilidad (Multi-AZ) detrás de un Application Load Balancer.
- Tolerancia a Fallos: Implementación de los patrones Circuit Breaker, Retry con Exponential Backoff y Timeout mediante la librería *Polly*. Si el Sistema Independiente de detalle del cliente presenta caídas o demora más de 3 segundos, el Circuit Breaker se abre y el sistema aplica *degradación elegante*: le muestra al usuario sus saldos y datos básicos desde el Core/Redis sin bloquear la pantalla.
- Auto-Healing (Autocuración): En Kubernetes (Amazon EKS) cada microservicio tiene configuradas sondas livenessProbe y readinessProbe. Si un contenedor se bloquea o agota memoria, el orquestador lo destruye y levanta un pod nuevo de forma transparente. El *Horizontal Pod Autoscaler (HPA)* agrega réplicas automáticamente en quincenas o fines de mes.
- Recuperación ante Desastres (DR): Esquema Warm Standby Multi-Región en us-east-2 (Ohio) con RPO < 1 minuto (mediante DynamoDB Global Tables y réplicas de lectura cross-region en RDS) y RTO < 15 minutos automatizado con health checks en Amazon Route 53 para conmutar el tráfico si cae la región completa.

6.3. Monitoreo y Excelencia Operativa

- Observabilidad Centralizada: Instrumentación con OpenTelemetry en todos los microservicios. Las métricas de infraestructura y negocio se recolectan en Prometheus y Grafana; los logs estructurados van a Amazon OpenSearch (ELK) con un Correlation-ID único por transacción; y el rastreo extremo a extremo se realiza con AWS X-Ray / Jaeger para detectar cuellos de botella en llamadas al Core.
- Excelencia Operativa: Infraestructura como código (IaC) con Terraform y pipelines CI/CD con despliegues *Blue/Green* o *Canary* para actualizar versiones sin ventana de mantenimiento.

6.4. Manejo y Optimización de Costos

- Ahorro en Cómputo: Uso de instancias reservadas (*Savings Plans*) para la carga base de operación 24/7 y nodos bajo demanda gestionados con *Karpenter* que solo se encienden en horas pico bancarias (9:00 a 19:00) y se reducen al mínimo en la madrugada.
- Descarga del Core Bancario: La caché Redis para clientes frecuentes absorbe cerca del 70% de las lecturas repetitivas de saldos y movimientos, reduciendo costos de procesamiento en el servidor Core.
- Ciclo de Vida de Auditoría (Storage Tiering): Los registros de auditoría se mantienen en caliente en DynamoDB durante 90 días para consultas inmediatas, y mediante reglas automáticas pasan a Amazon S3 Glacier Deep Archive para cumplir con los 5 a 7 años de retención legal a un costo mínimo por gigabyte.